5.4 Patrones de diseño

Proyecto: Monitor de Consumo Eléctrico — EnergyMonitor **Ficha:** 3145556 — Análisis y Desarrollo de Software (ADSO), SENA CIES Regional Huila **Versión:** 1.0

> **Nota de actualización (27/10/2026).** Este documento describe el diseño de la versión 1.0, cuando el backend era el monolito modular `energy-monitor-backend` (Spring Modulith). Ese diseño evolucionó: los siete contextos acotados son hoy **siete microservicios** detrás del API gateway `en-monitor-api-gateway` (8080) — seguridad 8082, dispositivos 8084, mediciones 8085, hogares 8086, notificaciones 8087, alertas 8088 y recomendaciones 8089 —, comunicados por REST interno (`/internal/**`) y eventos de RabbitMQ con outbox transaccional, sobre el mismo esquema MySQL compartido (37 tablas). El backend modular quedó retirado. Las capas hexagonales, los contextos y los patrones descritos aquí se conservan dentro de cada servicio; la arquitectura vigente, las rutas, los eventos y la configuración están en el manual técnico (secciones 9 a 13) y el modelo de datos vigente en `06-base-datos`.

1. Propósito

Un patrón de diseño es una solución reutilizable a un problema que aparece con frecuencia en el diseño de software. Aplicar un patrón no consiste en copiar su estructura, sino en reconocer **el problema** y elegir la solución que mejor encaja, sabiendo medir también su costo.

Esta sección documenta los patrones empleados en el proyecto «Monitor de Consumo Eléctrico», clasificados según los dos catálogos clásicos:

- **Patrones creacionales** — cómo se construyen los objetos.
- **Patrones estructurales** — cómo se componen los objetos.
- **Patrones de comportamiento** — cómo colaboran los objetos.

Cada patrón se documenta con cuatro elementos: el problema que resuelve, cómo se aplica en este proyecto (con la evidencia concreta en el código), qué beneficio aporta y qué costo introduce.

2. Resumen de patrones aplicados

#	Categoría	Patrón	Componente	Estado
1	Estructural	**Adaptador (Adapter)**	Backend y front-ends	Implementado
2	Estructural	**Puertos y adaptadores (Hexagonal)**	Backend	Implementado
3	Estructural	**Fachada (Facade)**	Backend	Implementado
4	Estructural	**Composición y agregación**	Modelo de datos	Implementado
5	Comportamiento	**Observador (Observer)**	Backend	Implementado con eventos
6	Comportamiento	**Estrategia (Strategy)**	Backend	Implementado
7	Comportamiento	**Cadena de responsabilidad**	Backend	Implementado
8	Comportamiento	**Cadena de filtros**	Backend y front-ends	Implementado
9	Comportamiento	**Comando (Command)**	Backend	Implementado
10	Creacional	**Método de fábrica (Factory Method)**	Backend	Implementado
11	Creacional	**Singleton**	Backend	Implementado por el contenedor
12	Creacional	**Constructor (Builder)**	Backend	Implementado
13	Creacional	**Almacén de objetos (Repository)**	Backend	Implementado
14	Creacional	**Prototipo (Prototype)**	Front-ends (in contextual)	Implementado

#	Categoría	Patrón	Componente	Estado
15	Comportamiento	**Proxy (Proxy)**	Backend	Implementado
16	De dominio	**Objeto de valor (Value Object)**	Backend	Implementado
17	De dominio	**Objeto de dominio (Entity)**	Backend	Implementado
18	De dominio	**Servicio de dominio (Domain Service)**	Backend	Implementado

3. Patrones estructurales

3.1 Adaptador (Adapter)

Problema. Un sistema debe trabajar con una interfaz que no diseñó: una base de datos, un servicio de correo, un bróker de mensajes o una API de otro módulo.

Aplicación en el proyecto. El backend implementa interfaces («puertos») en `application/port/out/` y provee clases «adaptador» en `adapter/out/` que las conectan con la tecnología concreta.

Puerto (interfaz)	Adaptador (implementación)	Tecnología oculta
`PasswordHasherPort`	`BCryptPasswordHasherAdapter`	BCrypt
`PasswordResetTokenHasherPort`	`PepperedHmacResetCodeHasher`	HMAC-SHA256 con pepper
`PasswordResetTokenHasherPort`	`Sha256TokenHasher`	SHA-256
`TokenGeneratorPort`	`SecureTokenGeneratorAdapter`	`SecureRandom`
`IdentifierGeneratorPort`	`SecureIdentifierGeneratorAdapter`	`SecureRandom`
`ResetAttemptLimiterPort`	`InMemoryResetAttemptLimiter`	Estructuras en memoria
`PushSenderPort`	`WebPushSender`	API de Web Push del navegador
`UserPersistencePort`	`UserPersistenceAdapter`	Spring Data JPA + Hibernate
`HomeApi` (de `home::api`)	`HomeApiImpl`	Contrato interno del contexto de hogares
`GoogleIdentityPort`	`GoogleOAuthClientAdapter`	API de Google
`ProfileImageFetcherPort`	`GoogleProfileImageFetcher`	Descarga de la imagen de Google
`PasswordResetDeliveryPort`	`EmailPasswordResetDeliveryAdapter`	SMTP
`EmailVerificationDeliveryPort`	`EmailVerificationDeliveryAdapter`	SMTP

Beneficio. El dominio no contiene ni una línea de código de BCrypt, SMTP ni SQL. Una prueba del caso de uso `AuthenticateUserService` sustituye `PasswordHasherPort` por un doble y se ejecuta sin contenedor de Spring.

Costo. Multiplica el número de clases: 64 puertos de salida para 64 contratos. Un módulo pequeño (por ejemplo `alert`, con una sola tabla) puede resultar desproporcionado frente al beneficio.

3.2 Puertos y adaptadores (Arquitectura hexagonal)

Problema. Acoplar el modelo de negocio al framework, a la base de datos o al protocolo de transporte hace imposible reutilizar el modelo y dificulta su verificación.

Aplicación en el proyecto. Cada módulo declara dos conjuntos de interfaces:

Tipo de puerto	Ubicación	Responsabilidad	Ejemplo
Puerto de entrada	`application/port/in`	Declara una operación de negocio que el mundo exterior puede solicitar	`CreateHome`, `RegisterMeasurement`
Puerto de salida	`application/port/out`	Declara una dependencia que el módulo necesita del exterior	`HomePersistencePort`

Regla verificada. El código del dominio (`domain/model`) sólo importa `java.*`. Los casos de uso de `security` tampoco importan Spring: se cablean con métodos `@Bean` explícitos, no con `@Service`.

> **Excepción documentada.** `HomeDeviceService` es el único caso de uso anotado con `@Service` y `@Transactional`. Es una clase heredada de una fase anterior que rompe la regla del módulo. Debe migrarse al cableado explícito de `DeviceServiceConfiguration` en la siguiente iteración.

Beneficio. Permite tres cosas que ninguna otra estructura permite aquí: 1. Probar los casos de uso sin servidor ni base de datos. 2. Sustituir una tecnología sin tocar el dominio (por ejemplo, cambiar MySQL por PostgreSQL). 3. Verificar la arquitectura: `ModularityTests` falla si un dominio importa algo que no le corresponde.

Costo. El número de archivos crece de forma notable: en el módulo `security` hay 14 entidades de dominio, 23 puertos de salida, 17 puertos de entrada y 19 DTO. La curva de aprendizaje para un desarrollador nuevo es el precio de esta estructura.

3.3 Fachada (Facade)

Problema. Un subsistema complejo debe exponer una interfaz simple de uso frecuente.

Aplicación en el proyecto. `SecurityLoginFlow` es la fachada del proceso de inicio de sesión: coordina cinco operaciones internas y devuelve un único resultado (`LoginOutcome`).

```
SecurityLoginFlow.login(email, password, ip)
  │
  ├──► AuthenticateUser.authenticate(...)      →  valida credenciales, estado y verificación
  ├──► CreateUserSession.create(...)           →  crea la sesión
  ├──► RefreshToken.root(...)                  →  genera la primera generación de refresh
  └──► JwtTokenIssuer.issue(...)               →  firma el access token
       │
       └──► LoginOutcome { accessToken, refreshToken, expiresIn, account }
```

El controlador `AuthController` no conoce ningún detalle de este proceso: llama a un solo método.

Otras fácadas del proyecto:

Clase	Simplifica
`SecurityFilterChainConfiguration`	El DSL de seguridad de Spring Security, que de otro modo ocuparía un método con más de cuarenta líneas
`SecurityCorsConfiguration`	La construcción de `CorsConfigurationSource`, con validaciones que lanzan `IllegalStateException` ante configuraciones inseguras
`HomeDeviceService`	Una sola clase que implementa seis interfaces de caso de uso del módulo de dispositivos

Beneficio. El proceso de autenticación puede cambiar por completo (por ejemplo, añadir autenticación multifactor) modificando una única clase.

Costo. La fachada puede convertirse en un «Dios» que concentra responsabilidades. `SecurityLoginFlow` se mantiene pequeño y sin estado precisamente para evitarlo.

3.4 Composición y agregación

Problema. Al modelar relaciones entre objetos, se debe elegir entre «pertenencia fuerte» (la parte muere con el todo) y «dependencia débil» (ambos pueden existir por separado).

Aplicación en el proyecto. La elección se hizo de forma explícita para cada relación del modelo de datos:

Relación	Tipo	Justificación
`Home 1 *-- 1 HomeThresholds`	**Composición**	Los umbrales no existen sin un hogar; si el hogar se elimina, sus umbrales desaparecen
`User 1 *-- 1 UserConfiguration`	**Composición**	La configuración es un atributo de la cuenta, no una entidad con vida propia
`UserSession 1 *-- 1..* RefreshToken`	**Composición**	La sesión posee sus generaciones; revocar la sesión invalida todas

Relación	Tipo	Justificación
`Home "*" o-- "1" HomeType`	**Agregación**	El catálogo es compartido: un tipo de hogar sirve a muchos y no pertenece a ninguno
`Device "*" o-- "1" ApplianceType`	**Agregación**	El catálogo de electrodomésticos es compartido por todos los dispositivos
`Home 1 -- "0..*" UserHome`	**Asociación**	La pertenencia se conserva aunque el hogar se elimine (borrado lógico)
`UserHome` con PK compuesta	**Clave compuesta**	La pertenencia se identifica por el par usuario–hogar, no por un identificador propio

Beneficio. La elección queda reflejada en el esquema (llaves foráneas `NOT NULL`, unicidad en `home_thresholds.home_id`) y no depende de la intención del diseñador.

Costo. Elegir mal la cardinalidad obliga a migraciones costosas. Por eso la regla se documentó en el diagrama de clases.

4. Patrones de comportamiento

4.1 Observador (Observer)

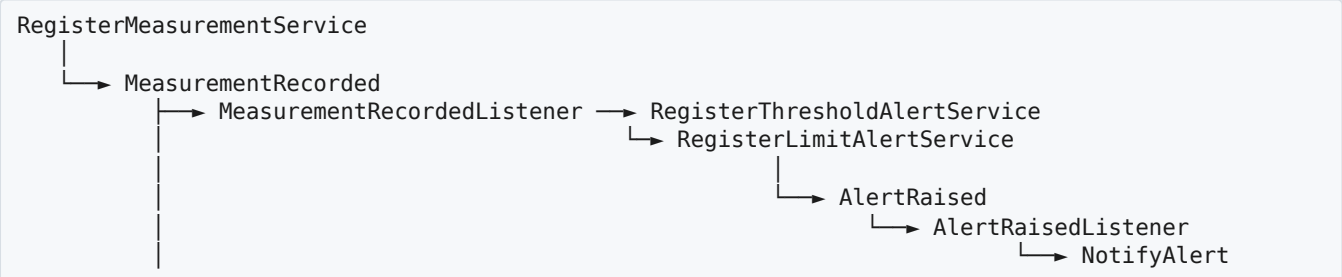
Problema. Cuando ocurre un evento, hay que avisar a varios destinatarios sin que el emisor los conozca ni dependa de ellos.

Aplicación en el proyecto. Es el patrón central del flujo reactivo del sistema.

Participantes en la cadena de mediciones:

Rol del patrón	Clase	Módulo
Sujeto	`MeasurementEventAdapter` (publica `MeasurementRecorded`)	`measurement`
Observador	`MeasurementRecordedListener` (`@EventListener`)	`alert`
Sujeto secundario	`AlertEventAdapter` (publica `AlertRaised`)	`alert`
Observador	`AlertRaisedListener` (`@Async`)	`notification`

Cadena completa:



Variante del patrón — eventos con nombre. La implementación usa eventos con nombre (`ApplicationEventPublisher` y `@EventListener`) en lugar de la lista de observadores del patrón canónico. La diferencia es relevante:

Aspecto	Observer canónico	Implementación del proyecto
Suscripción	El sujeto mantiene la lista	El sujeto sólo publica
Acoplamiento	El sujeto conoce a sus observadores	Ninguno: quien publica ignora quién escucha
Consecuencia	Añadir un observador obliga a modificar el sujeto	Añadir un observador no toca al sujeto
Pérdida	Un fallo del observador afecta al emisor	Un fallo del observador no impide la entrega

> **Decisión documentada en el código.** La clase `MeasurementRecordedListener` incluye el comentario: *«the class diagram's* `AlertObserver`*»*. Es decir: el diagrama de clases describe el patrón Observer, y el código lo

> materializa con eventos en proceso, que es la variante con menos acoplamiento.

Recomendaciones (implementado). El módulo `recommendation` no observa las mediciones: `RecommendationScheduler` ejecuta cada hora `EvaluateRecommendationsService`, que lee el consumo mediante puertos de salida, aplica las reglas de ahorro y, por cada recomendación nueva, publica `RecommendationCreated` mediante `RecommendationEventAdapter`; `notification` lo escucha con `RecommendationCreatedListener`.

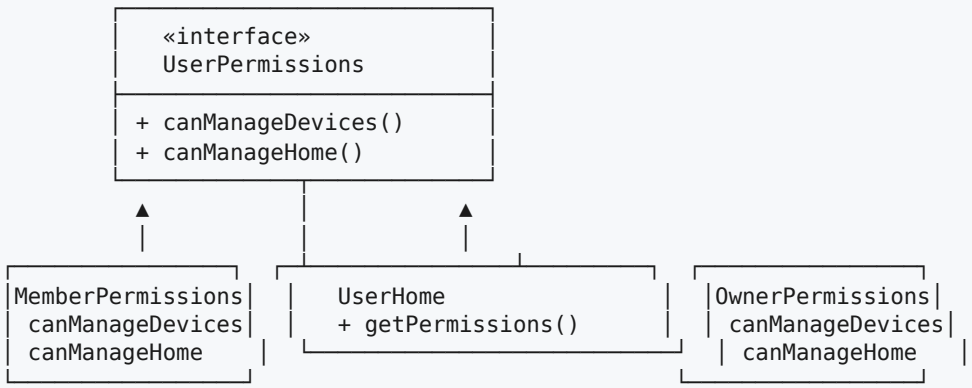
Beneficio. El módulo `measurement` no sabe que existen alertas. Se puede añadir otro observador sin modificar una línea del módulo de mediciones.

Costo. Los eventos en proceso **no son persistentes**: si el proceso se reinicia entre la publicación y la escucha, el evento se pierde. Para el sistema actual es aceptable (las mediciones se reenvían cada 60 segundos), pero un flujo que requiera garantías de entrega necesita una cola de mensajes.

4.2 Estrategia (Strategy)

Problema. Un comportamiento debe variar según el tipo de objeto, sin llenar la clase de ramas `if`/`switch` ni forzar al llamador a conocer todas las variantes.

Aplicación en el proyecto. La autorización dentro de un hogar varía según el rol (`OWNER` o `MEMBER`).



Evidencia en el código. Las tres clases existen en `home/domain/model`, y `UserHome.getPermissions()` devuelve la implementación que corresponde a su `role`. `DeviceExceptionHandler` y `HomeExceptionHandler` lanzan `NotHomeOwnerException` cuando la estrategia del propietario no permite la operación.

> **Corrección importante.** La nota del diagrama de clases entregado afirma que estas tres clases «no existen todavía en el código Java» y que la autorización real se resuelve por una cadena RBAC genérica. **Esa nota está desactualizada:** las tres clases están implementadas. Conviene corregir el diagrama antes de la entrega (ver `05.3-modelo-de-clases.md`, sección 10).

Diseño complementario — doble sistema de roles. El sistema tiene **dos** mecanismos de autorización que no deben confundirse:

Mecanismo	Alcance	Resolución	Ejemplo
Rol por hogar (`Role`)	Un usuario dentro de un hogar concreto	`user_home.role` → `UserHome.getPermissions()`	Solo el propietario puede vincular dispositivos o expulsar miembros
RBAC global (`CheckPermission`)	Permisos funcionales del sistema	Consulta en vivo de la cadena `system_role` → `user_system_role` → `system_role_permission` → `permission`	Permisos como `METER.READ` o `ALERT.READ`

Beneficio. Añadir un tercer rol (por ejemplo, `INVITADO`) consiste en crear una clase que implemente `UserPermissions` y asignarla desde `UserHome.getPermissions()`. Ninguna otra clase del sistema cambia.

Costo. La cantidad de clases crece con el número de variantes. Con dos variantes, el beneficio es menor que el costo; con cinco o más, la estrategia se justifica claramente.

4.3 Cadena de responsabilidad (Chain of Responsibility)

Problema. Una petición debe recorrer una secuencia de handlers, y cada uno decide si la atiende o la pasa al siguiente.

Aplicación en el proyecto. La autorización de acceso al bróker MQTT en `BrokerAuthController.authorize(username, topic, access)` recorre los niveles de acceso que el bróker solicita:

```
switch (request.access()) {
    case 1  -> /* solo lectura    */ break;
    case 2  -> /* solo escritura  */ break;
    case 3  -> /* lectura y escritura */ break;
    case 4  -> /* suscripción     */ break;
    default -> /* denegar         */ break;
}
```

Beneficio. Cada nivel de permiso se valida con una regla distinta; añadir un nivel no obliga a modificar la lógica de los anteriores.

Costo. El flujo de control se vuelve implícito: para saber qué valida un nivel hay que leer el `switch` completo.

4.4 Cadena de filtros (Filter Chain)

Problema. Una petición debe atravesar una secuencia de comprobaciones antes de llegar al recurso.

Aplicación en el proyecto. En el backend, `SecurityFilterChainConfiguration` define una cadena única de Spring Security:

Orden	Componente	Función
1	`BrokerAuthSecurityConfiguration` (`@Order(1)`)	Protege `/internal/mqtt/**` restringiendo por red autorizada
2	`SecurityCorsConfiguration`	Aplica la política de CORS y responde al *preflight* `OPTIONS`
3	`BearerTokenAuthenticationFilter` (de Spring Security)	Valida el JWT `RS256` y establece el contexto de seguridad
4	`JwtAuthenticationEntryPoint`	Responde `401` con el formato `ApiError`
5	`JwtAccessDeniedHandler`	Responde `403` con el formato `ApiError`
6	`AuthorizationFilter`	Aplica las reglas de autorización por endpoint

En los front-ends, la misma idea aparece en los interceptores del cliente HTTP:

Capa	Componente	Función
Petición	`httpClient` — interceptor de petición	Añade `Authorization: Bearer`, omite endpoints públicos y renueva el token si expiró
Respuesta	`httpClient` — interceptor de respuesta	Ante `401` intenta renovar una sola vez y reintenta la petición

Beneficio. Cada filtro tiene una única responsabilidad y puede probarse por separado.

Costo. El orden de los filtros es implícito; cambiarlo puede romper la seguridad sin que ninguna prueba lo detecte.

4.5 Comando (Command)

Problema. Una operación necesita transportar datos de entrada y resultados de salida sin acoplar la interfaz al modelo de dominio.

Aplicación en el proyecto. Cada puerto de entrada expone un método que recibe un *record* inmutable y devuelve otro.

Componente	Ejemplo
Comando	`RegisterUserCommand`, `CreateHomeCommand`, `RegisterMeasurementCommand`
Consulta	`CheckPermissionQuery`
Resultado	`AuthenticationResult`, `HomeResult`, `AlertResult`, `RefreshSessionResult`

Diferencia explícita entre comando y consulta:

Tipo	Verbo	Efecto	Ejemplo
Comando	Imperativo: `register`, `create`, `update`, `delete`	Modifica el estado	`CreateHomeCommand`
Consulta	Interrogativo: `find`, `list`, `get`, `check`	Sólo lee	`CheckPermissionQuery`

Beneficio. Los registros de comando son inmutables y se pueden construir en cualquier capa. Los casos de uso se prueban construyendo el comando directamente, sin construir DTOs HTTP.

Costo. Multiplica los tipos: para un solo caso de uso existen a menudo cuatro archivos (puerto, comando, resultado y servicio).

5. Patrones creacionales

5.1 Método de fábrica (Factory Method)

Problema. Una clase debe crear objetos sin conocer las clases concretas que va a instanciar, o debe garantizar que sus invariantes se respeten en el momento de la construcción.

Aplicación en el proyecto. Los agregados ofrecen factorías estáticas que garantizan la validez de la instancia creada:

Agregado	Factorías	Invariante garantizado
`Alert`	`threshold()`, `limit()`, `connectivity()`, `device()`	Una alerta de umbral tiene nivel de consumo y medición; una de conectividad, no
`RefreshToken`	`root()`, `childOf()`	La primera generación no tiene padre; las siguientes sí, y heredan la familia
`Device`	`create()`, `relinked()`, `edited()`, `withApiKey()`	Un dispositivo siempre tiene código único y clave de integración
`DeviceStatusLog`	`create()`	El registro tiene estado y última conexión
`DeviceHome`	`of()`	La vinculación pertenece a un dispositivo y a un hogar válidos
`LoginErrorLog`	`userNotFound()`	Un intento fallido sin usuario conocido se registra sin identificador de usuario
`AuditLog`	`login()`	La acción auditada cumple el formato esperado
`Email`, `PasswordHash`	`of()`	El valor es válido según su formato

Beneficio. Un cliente que quiera una alerta de límite **no puede** construirla con valores inconsistentes: la única vía es la factoría correcta. La invariante no depende de la disciplina del programador.

Costo. Cada nuevo tipo de alerta exige añadir una factoría. Con diez tipos, el agregar se vuelve costoso; para cuatro tipos, es manejable.

5.2 Singleton

Problema. Una clase debe tener una única instancia compartida por toda la aplicación.

Aplicación en el proyecto. El contenedor de Spring gestiona el ciclo de vida; todos los beans son singletons por defecto. Los casos que lo requieren de forma explícita están declarados:

Bean	Clase	Motivo
`clock()` (`@Primary`)	`Clock.systemUTC()`	Fuente única de tiempo; permite pruebas con reloj fijo
`securityClock()`, `mailDeliveryClock()`	`Clock`	Relojes independientes por módulo
`jwtEncoder()`, `jwtDecoder()`, `jwtKeyId()`	Nimbus	El par de claves RSA se lee una sola vez del sistema de archivos
`passwordResetExecutor()`	`ThreadPoolExecutor`	Un único grupo de hilos acotados para la recuperación de contraseña
`mqttClientFactory()`	`MqttPahoClientFactory`	Una conexión al bróker, no una por mensaje

Beneficio. El reloj único es lo que permite escribir pruebas deterministas: si el tiempo se leyera del sistema, ninguna prueba de expiración de token sería reproducible.

Costo. El estado global en beans singleton complica las pruebas concurrentes y puede introducir condiciones de carrera si un bean acumula estado mutable.

5.3 Constructor (Builder)

Problema. Un objeto debe construirse con muchos parámetros opcionales, y la firma con todos ellos sería ilegible.

Aplicación en el proyecto. El patrón se manifiesta en dos formas:

Forma	Dónde	Ejemplo
Records como constructores	Backend	`RegisterUserCommand(email, password, person, profileImage, ipAddress, zone)` — el compilador exige los campos obligatorios y el orden es explícito
Cadenas de texto con plantilla	Backend	`NotifyAlert.sendMail(...)` construye el cuerpo con bloques de texto y `.formatted(...)`

Beneficio. Con los records, el compilador garantiza que no se olvide ningún campo obligatorio ni se pase un argumento en la posición equivocada.

Costo. Los records con muchos campos siguen siendo incómodos de construir en las pruebas, donde se recurre a constructores auxiliares (`UseCaseFixtures`, `RefreshTokenFixtures`).

5.4 Almacén de objetos (Repository)

Problema. La aplicación necesita consultar y persistir objetos sin conocer el detalle del almacenamiento.

Aplicación en el proyecto. El proyecto usa el patrón en dos niveles:

Nivel	Tecnología	Cantidad
PUERTO de salida (dominio)	Interfaces `*PersistencePort`	64 puertos de salida
ADAPTADOR (infraestructura)	`*Repository extends JpaRepository`	29 interfaces

Ejemplo completo del camino de una consulta:

```
AlertController.getAlerts(homeId, status)
   │
   ▼  ListAlertsService  (caso de uso)
AlertListAlertsPort  ──►  ListAlertsService
   │                              │
   │                              ▼  puerto de salida
   │                    AlertPersistencePort
   │                              │
   │                              ▼  adaptador
   │                    AlertPersistenceAdapter
   │                              │
   │                              ▼  repositorio Spring Data
```

```
   │                  AlertRepository  ──►  Hibernate  ──►  tabla `alert`
   ▼
AlertResponse  (DTO JSON)
```

Beneficio. El caso de uso se prueba con un doble de `AlertPersistencePort`; la prueba real contra MySQL se hace en una capa distinta.

Costo. Para tablas muy simples, el nivel de puerto es innecesario. Es el patrón que más sofre el coste de la arquitectura hexagonal.

5.5 Prototipo (Prototype) — instantiation en contextual

Problema. Un objeto debe crearse a partir de una plantilla existente y luego modificarse.

Aplicación en el proyecto. Ambos front-ends usan `createPortal` de React DOM para renderizar el modal de cuenta sobre el `document.body` sin desmontar el resto del árbol:

```
Sidebar (dentro de MainLayout)
   │
   └──► createPortal(<Account/>, document.body)
          └──► Modal de cuenta por encima de toda la aplicación
```

Variante en el móvil. El estado del menú lateral se clona en cada apertura animando una copia del estilo mediante `Animated.timing` con `useNativeDriver: true`, en lugar de reconstruir el componente.

Beneficio. Evita recrear la estructura completa del modal y garantiza que se superponga a cualquier contenido, incluso si el contenedor tiene `overflow: hidden`.

Costo. Los portales se escapan del flujo visual del árbol de React: eventos y estilos heredados deben resolverse explícitamente.

6. Patrones adicionales del proyecto

6.1 Proxy dinámico

Problema. Una clase necesita incorporar comportamiento transversal —en este caso, una transacción— sin que su código lo mencione.

Aplicación en el proyecto. `SecurityApplicationWiring` construye un `ProxyFactory` programático con un `TransactionInterceptor` en lugar de anotar los casos de uso con `@Transactional`:

```
private <T> T transactional(Class<T> type, T target, T port) {
    ProxyFactory proxyFactory = new ProxyFactory();
    proxyFactory.setTarget(target);
    proxyFactory.setInterfaces(port);   // sólo la interfaz del puerto de entrada
    proxyFactory.addAdvice(transactionInterceptor);
    return (T) proxyFactory.getProxy();
}
```

Detalle relevante. El proxy implementa **únicamente la interfaz del puerto de entrada**. Esto significa que una llamada desde otro punto del sistema a la implementación concreta **no** pasa por la transacción. Es una barrera de seguridad arquitectónica, no un descuido.

Beneficio. La frontera transaccional es explícita y está probada: `SecurityTransactionBoundaryTest` verifica con `AopUtils.isAopProxy` qué casos de uso son transaccionales y `SecurityTransactionRollbackTest` verifica el comportamiento ante fallo.

Costo. La corrección es invisible para quien lee el código: un caso de uso no muestra nada que indique dónde empieza y termina su transacción.

6.2 Objeto de valor (Value Object)

Problema. Ciertos datos no tienen identidad: son valores. Tratarlos como cadenas sueltas permite que un correo mal formado llegue hasta la base de datos.

Aplicación en el proyecto.

Objeto de valor	Invariante	Cómo se garantiza
`Email`	Formato válido, no vacío	Factoría estática `Email.of()`
`PasswordHash`	No vacío, producido por un hasher	Factoría estática `PasswordHash.of()`
`Preconditions`	Reglas de invariantes de cada agregado	Clase `final` con visibilidad de paquete en cada módulo
`Measurement`	Magnitudes inmutables tras el registro	Sin setters

Propiedades garantizadas: igualdad por valor, inmutabilidad e intercambiabilidad.

Beneficio. `User.changeEmail(newEmail: Email)` no puede recibir una cadena arbitraria. La validación está en la construcción, no en el uso.

Costo. Un objeto de valor no puede tener identidad, por lo que no se persiste por separado. Para datos que a veces son valor y a veces identidad (una dirección de correo que también es llave), la decisión requiere criterio.

6.3 Objeto de dominio (Entity)

Problema. Algunos datos tienen identidad y un ciclo de vida; deben garantizar sus invariantes a lo largo de ese ciclo.

Aplicación en el proyecto. `User`, `Home`, `Device`, `Alert` y `RefreshToken` son objetos de dominio: su comportamiento vive en la clase.

Objeto	Invariante protegida por el comportamiento propio
`User`	`canAuthenticate()` sólo es verdadero si está `ACTIVE` y verificado
`User`	`incrementFailedLoginAttempts()` nunca deja el contador en negativo
`UserSession`	`isActive(now)` es falso si la sesión está revocada o cerrada
`RefreshToken`	`isUsable(now)` es falso si está rotado, revocado o expirado
`Home`	`addUser(u, role)` valida que el usuario no esté ya vinculado

Beneficio. La invariante se mantiene sin que cada llamador recuerde validarla.

Costo. El objeto acumula responsabilidades. Si se modela demasiado comportamiento en el dominio sin control, la clase se vuelve difícil de probar; el equilibrio lo marca el principio «el dominio no sabe nada de infraestructura».

6.4 Servicio de dominio (Domain Service)

Problema. Un comportamiento que atraviesa varios agregados no pertenece naturalmente a ninguno de ellos.

Aplicación en el proyecto. `RecommendationRules` es la única clase en `domain/service/`. Analiza mediciones de todos los dispositivos de un hogar y devuelve sugerencias: no pertenece a `Measurement` (que sólo conoce una lectura), ni a `Device` (que no ve el conjunto), ni a `Recommendation` (que es el resultado).

Propiedad	Valor
Dependencias	Ninguna
Entrada/salida	Ninguna
Reloj	Se recibe como parámetro (`Instant`, `ZoneId`)

Beneficio. Las reglas son puras y deterministas: la misma entrada produce siempre la misma salida, en cualquier zona horaria y sin necesidad de un contenedor de Spring.

Costo. Un servicio de dominio sin dependencias es, estructuralmente, un conjunto de funciones. La ventaja aparece cuando las reglas crecen; con pocas reglas, una función en el caso de uso bastaría.

7. Patrones en los front-ends

Los dos front-ends replican el mismo conjunto de patrones de presentación, lo que garantiza que la experiencia sea homogénea.

#	Patrón	Aplicación en la web	Aplicación en el móvil
1	**Design tokens**	Variables CSS en `:root`, reasignadas por tema	Sistema de tokens en `constants/` y paletas en `theme/`
2	**Tema dinámico**	`ThemeProvider` escribe variables CSS en el elemento raíz	`createThemeStyles` reconstruye las hojas al cambiar de paleta
3	**Composición sobre herencia**	`children` en `Input`, `Header`, `Card`	`children` en `Header`, `Modal`, `Alert`
4	**Variantes**	`variant` en `Button`, `Card`, `Input`, `Badge`	`variant` en `Button`, `Card`, `Input`, `Modal`, `Alert`
5	**Elevación de estado**	`Context` + `useSyncExternalStore` para la sesión	`Context` + `useSyncExternalStore` para la sesión
6	**Composición de contextos**	`useOutletContext()` comparte el recurso con la ruta hija	Contextos anidados en `_layout.tsx`
7	**Capa de datos única**	`services/` con un solo cliente HTTP	`services/` con un solo cliente HTTP
8	**Máquina de estados**	Pasos del asistente de vinculación Bluetooth	Pasos del asistente de vinculación
9	**Estados declarativos**	`loading` / `error` / `empty` / `content`	`loading` / `error` / `empty` / `content`
10	**Cola única (single flight)**	Renovación de token serializada con Web Locks	Renovación de token serializada con una promesa de módulo

7.1 Detalle destacado — Sistema de temas móvil

El caso más interesante del diseño de interfaces es el tema en React Native. React Native no tiene variables CSS, por lo que la web y el móvil resolvieron el mismo problema de forma distinta:

Enfoque	Solución
Web	Variables CSS escritas en `:root`; el navegador recalcula todo solo
Móvil	`createThemeStyles` envuelve `StyleSheet.create` en un `Proxy` que memoriza la hoja por paleta y la reconstruye cuando cambia

Problema adicional documentado en el código. Con el compilador de React activo, el render de cada componente se memoiza según sus dependencias reactivas. La paleta del tema no está entre ellas, así que una pantalla puede reutilizar su JSX cacheado y conservar los estilos anteriores.

Solución adoptada. Las 13 rutas del móvil renderizan con `key={currentTheme.id}`. La clave fuerza el remontaje del componente y garantiza el repintado completo. Es una decisión consciente, documentada en `ThemeContext.tsx`, que resuelve una limitación real de la combinación entre el compilador de React y los tokens de tema.

8. Decisiones sobre aplicar o no un patrón

No todos los problemas requieren un patrón. Se registran las decisiones de **no** aplicar uno:

Situación	Decisión	Justificación
Estado global en los front-ends	**Context, no Redux ni Zustand**	El estado global es reducido (sesión, tema, hogares, notificaciones). Redux añadiría ceremonia y boilerplate sin beneficio. `zustand` (web) y `@reduxjs/toolkit` con `react-redux` (móvil) figuran en los `package.json`, pero el código fuente no los importa
Gráficas	**Biblioteca especializada, no patrón propio**	Recharts y `react-native-chart-kit` resuelven el problema; escribir un motor de gráficas propio no aporta valor
Validación de formularios	**Zod, no un validador propio**	El esquema declarativo cubre el caso y produce tipos
Caché de datos en los front-ends	**Sin caché, con sondeo**	El volumen de datos es bajo y la carga bajo demanda es suficiente. Una caché exigiría invalidación, que es más compleja que el problema que resuelve
Paginación	**No implementada**	Ninguna tabla del sistema crece actualmente más allá de decenas de filas

9. Trazabilidad de los patrones con los requisitos

Patrón	Requisito o restricción que satisface
Puertos y adaptadores	RNF-MAN (mantenibilidad) — los casos de uso se prueban sin infraestructura
Límites de módulo ejecutables	RNF-MAN — la arquitectura se verifica automáticamente
Observador	RF-ALE (alertas automáticas) — desacopla medición de alerta
Estrategia	RF-HOM-05 (permisos de usuario) — autorización por rol
Método de fábrica	RF-ALE — invariantes de las alertas en el momento de crearlas
Cadena de filtros	RNF-SEG (seguridad) — controles de autenticación y autorización
Almacén de objetos	RNF-MAN — desacopla el dominio del esquema SQL
Objeto de valor	RNF-SEG (seguridad) — validación en la construcción, no en el uso
Design tokens	RNF-USA (usabilidad) — coherencia visual y consistencia entre plataformas
Composición de contextos	RNF-USA — flujo de navegación predecible
Estados declarativos	RNF-USA — mensajes claros ante error y estado vacío

10. Artefactos relacionados

Artefacto	Ubicación
Modelo de clases con los patrones	`05.3-modelo-de-clases.md`
Diagrama de clases (PlantUML)	`assets/diagramas/Diagrama_clases.wsd`
Modelo arquitectónico	`05.1-modelo-arquitectonico.md`
Reglas arquitectónicas verificables	`05.1-modelo-arquitectonico.md`, sección 8
Pruebas que verifican los límites	`energy-monitor-backend/src/test/java/com/energymonitor/ModularityTests.java` (repositorio retirado el 28/10/2026; vale para la versión 1.0, hoy cada servicio es un repositorio y un despliegue independiente, de modo que el límite entre contextos lo marca el propio servicio)